

Agentic Fuzzing without coverage

Target: [parson](#) (JSON, C) @ ba29f4e.

Grammar: grammars-v4 json/JSON.g4 @ e1c222f.

Result: 5 iterations, 2,500 inputs, 0 crashes, \$0.86 (91,927 tokens) of a \$5 budget.

The problem

Fuzzing a parser is easy to do badly. Throw random bytes at a JSON library and nearly every input gets crushed in the first few characters, so the tokenizer gets tested thousands of times, and the real parser almost never does. The usual fix is to generate inputs already shaped like the format, which is what a formal grammar gives you for free.

In this assignment, I wasn't allowed to write that generator myself. An LLM (Large Language Model) had to read the grammar and write the Hypothesis strategy. I'd run its output through a sanitizer build of the real C library and feed the measurements back so it could revise, for five rounds or five dollars, whichever ran out first. I also couldn't instrument the target for coverage, which is normally how you'd know if a generator got better. So my actual problem wasn't whether an LLM could write a grammar-based generator. It could, on the first try. It was what to show in round two.

Design

Grammar and adaptations: The 77-line JSON grammar goes into the prompt whole, because summarizing risks quietly dropping a production. The Grammar defines JSON as an idea. `parson` is one programmer's actual implementation of it. Wherever those two disagree is exactly where real code had to be written to handle the difference. Rather than guess, I measured it: 33 boundary inputs plus a bisection against the sanitizer build (`grammar/json-parson/ADAPTATIONS.md`). `parson` accepts things the grammar forbids (a trailing comma, anything after the first value) and rejects things it allows (duplicate keys, lone surrogates, exponents that overflow a double). It caps nesting at 2049 for arrays and 2048 for objects. That one-level split isn't noise, it holds on every run. They go in beside the grammar, because a generator emitting only grammar-legal text never reaches the superset paths.

Harness and build: Not every failure is a bug. Rejecting bad input is the parser working correctly, so the real challenge is drawing the line between that and an actual crash. The harness reads `stdin`, calls `json_parse_string`, and exits 0 when a value comes back, 1 on a clean `NULL`, 2 if the harness itself fails. Anything else, a fatal signal or sanitizer abort, is a real bug. The build compiles with `ASan` and `UBSan`, plus `-fno-sanitize-recover=all`, so the moment something undefined happens, the process dies immediately instead of silently limping forward. `ASan`'s default exit code is 1, the same code the harness uses for a clean rejection, so I set `ASAN_OPTIONS=abort_on_error=1` to keep every real memory bug from being misfiled as ordinary invalid input. Timeouts get 5 seconds and are triaged as crashes, since a parser that hangs is a denial-of-service bug, not a pass.

The loop and its signal: Each round prompts the model, saves the module, sanity-checks a dozen examples to catch one that won't even import, runs 500 inputs through the harness, and hands the summary back. In place of coverage, I tracked three things instead: how often inputs got accepted, which grammar rules were being exercised (each strategy self-reports this), and nesting depth. Those three signals were chosen because together they cover the usual failure modes of a grammar-derived generator: it bounces off the tokenizer, it silently never reaches some rules, or its recursion is cosmetic and everything comes out flat. Since none of them require touching the target, they come free and stay inside the rules.

Findings

`parson` survived all 2,500 inputs under `ASan` and `UBSan` with zero crashes. That isn't a broken pipeline: the crash path was validated first against a toy parser with three planted bugs, all caught, deduplicated, and minimized correctly (`spine_check/`, 6/6).

Iteration	Acceptance	Max depth	What drove the change
0	45.0%	8	seed from grammar + measured gaps
1	40.2%	7	dropped an unrecognized production label, widened leaf budget
2	58.6%	2051	fixed the depth instrumentation bug, added mid-range nesting
3	53.2%	2051	nested duplicate keys, block comments, more invalid UTF-8
4	52.6%	2052	exact wall depths, alternating array/object nesting

The interesting part is iteration 2. The round before shipped a probe aimed at 2,000+ levels of nesting, and the depth histogram came back topping out at 7. From that single number the model worked out that its `production()` call was wrapping the whole document instead of each level, rewrote it with an `ExitStack` entering one context per level, and depth jumped to 2051. That is exactly the "recursive generator that secretly flattens" failure the assignment warns about, caught by the signal.

The assignment mentions that when this exact exercise was trialed on parson before, it took five iterations to go from finding nothing to finding crashes consistently. Mine found none, and I think the signal is why. Depth was the most legible number I gave the model, it moved dramatically once fixed, and so three of five rounds went into pushing it further. But nesting depth is the one dimension parson explicitly defends, with a `MAX_NESTING 2048` guard that refuses cleanly instead of blowing the stack. The loop optimized hard toward a wall the author had already built. Meanwhile acceptance sat between 40% and 59% every round, my summary called that "healthy" every round, and a signal that keeps reporting "healthy" creates no pressure to change anything. The proxy worked as designed.

Still under-tested: the harness only calls `json_parse_string`, so `json_parse_file`, the `_with_comments` entry point, and the serialization path never ran. The strongest remaining suspect for a hidden memory bug is invalid UTF-8 inside strings, since parson does no encoding checks whatsoever, but that path only showed up in roughly 11% of inputs from iteration 3 on. The same gap shows up in nesting depth: most inputs sit either near zero or right at the wall, so the middle range was barely tested.

Challenges

The first real run crashed with a `JSONDecodeError`. It looked like a parser bug, but it wasn't: my token limit was too low, and the response got cut off mid-string once thinking and a full strategy module ate the budget. Raising the limit and checking the stop reason before parsing fixed it, at the cost of one wasted call.

Three judgment calls are worth explaining. First, deduplication: I group crashes by hashing their top three stack frames, after stripping out addresses and library noise. It was never tested against a real JSON crash here, since parson never crashed, but the same method was tested on a real crash in the secondary TOML report, where it revealed a real weakness: it over-counts stack-overflow bugs, splitting one root cause into four signatures. Second, timeouts: a hang counts as a crash, the same as a sanitizer abort. A parser that never terminates is a denial-of-service bug whether a sanitizer ever fires. Nothing here came close to timing out. Third, the input buffer is freed before the parse tree on purpose, so a retained pointer would show up as a real use-after-free rather than stay hidden. Nothing did.

With more time I'd widen the harness past a single entry point and try differential testing against a second JSON parser: disagreement is a denser signal than crashing, and catches correctness bugs a sanitizer never will. With coverage available, the failure above wouldn't have happened at all: it would have shown iterations 2 through 4 re-walking the same lines at ever-greater depth, and the pressure to look elsewhere would have arrived on round three instead of in this paragraph.

Artifacts under `json-parson/`: `grammar/`, `target/`, `strategies/`, `logs/`, `crashes/`.